

سند نقشه کلان و بلوپرینت معماری سیستم

ERP SaaS System Architecture Blueprint

نسخه: ۱٫۰ (Freeze)

وضعیت - Architectural Decision Document (ADD): مصوب و نهایی

مرجع فنی: اکوسیستم ۱۵+ Next.js , ۱۶+ PostgreSQL , ۱۱.x Laravel

=====

=====

۱. هدف سند

این سند ساختار کلان، لایه‌بندی زیرساختی و اصول یکپارچه‌سازی پلتفرم چندمستأجری HamarehERP را تبیین می‌کند. هدف از این بلوپرینت، ارائه‌ی یک مرجع فنی واحد (Single Source of Truth) برای چگونگی چیدمان، تعامل و مرزبندی اجزای نرم‌افزاری در لایه‌های دیتابیس، بک‌آند و فرانت‌آند است تا مقیاس‌پذیری طولانی‌مدت پلتفرم تضمین شود .

=====

=====

۲. لایه‌بندی کلان پلتفرم (The ۴-Layer Architecture)

معماری سیستم به صورت مطلق از یک ساختار چهار لایه‌ای مجزا تبعیت می‌کند. هرگونه تداخل یا نشت منطقی بین این لایه‌ها ممنوع است :

لایه ۱: پلتفرم تجاری (SaaS Business Core)

وظایف: ثبت‌نام مستأجران جدید، مدیریت پلن‌های اشتراک، سیستم صدور فاکتور و بیلینگ پلتفرم، احراز هویت اولیه کاربران سراسری و مدیریت لایسنس‌ها .

لایه ۲: لایه ابزارها و ادمین پلتفرم (SaaS Admin & Tools)

وظایف: پنل مدیریت مرکزی برای ادمین‌های سیستم HamarehERP جهت مانیتورینگ سلامت سیستم، مدیریت شکست رویدادها (Dead Letter Queue) ، پشتیبان‌گیری، ارتقای دیتابیس‌ها و اعمال محدودیت‌های مصرف (Rate Limiting).

لایه ۳: لایه سازمان و هویت هسته (Core Identity & Organization)

وظایف: مدیریت فیزیکی کاربران متصل به مستأجرها، نگاشت نقش‌ها (RBAC) ، کنترل محدوده‌های

سازمانی (Company, Branch, Department) و ساختار درختواره دسترسی داده‌ها (Scope Driven Access).

لایه ۴: ماژول‌های عمودی و بیزینسی (Vertical ERP Modules)
وظایف: پیاده‌سازی لاجیک و منطق کسب‌وکار ماژول‌های حسابداری مالی، انبارداری، زنجیره تامین، تولید، فروش و موتور گردش کار پلتفرم .

=====

=====

۳. تفکیک فیزیکی پایگاه‌داده (Physical Database Separation)
به منظور حفظ امنیت مطلق اطلاعات و جلوگیری از هرگونه ریسک نفوذ، دیتابیس لایه پلتفرم باید به صورت فیزیکی از دیتابیس عملیاتی مستأجران جدا باشد :

۳.۱. پایگاه‌داده مستقل لایه ۱: (SaaS Core DB)
تمامی داده‌های لایه ۱ و ۲ شامل جدول کل مستأجران، تراکنش‌های مالی پلتفرم و اطلاعات لایسنس‌ها در یک پایگاه‌داده فیزیکی مجزا به نام hamareh_saas_core نگهداری می‌شوند .

۳.۲. پایگاه‌داده مشترک لایه عملیاتی: (Tenant Operational DB)
تمام داده‌های لایه ۳ و ۴ (ماژول‌های ERP) در یک پایگاه‌داده مشترک مجزا به نام hamareh_erp_tenants ذخیره می‌گردند. تفکیک داده‌ها در این لایه بر اساس سند شماره ۱ (PostgreSQL RLS) اعمال می‌شود. ماژول‌های ERP تحت هیچ شرایطی حق اتصال مستقیم یا کوئری زدن به دیتابیس پلتفرم (SaaS Core DB) را ندارند .

=====

=====

۴. معماری بک‌آند (Modular Monolith with Domain-Driven Design)
ساختار کدهای بک‌آند در فاز اول پروژه به صورت Modular Monolith مبتنی بر اصول DDD پیاده‌سازی می‌گردد :

۴.۱. استقلال کدهای ماژول‌ها: کدهای پروژه نباید در پوشه‌های پیش‌فرض لاراول مخلوط شوند.
سیستم با استفاده از پکیج‌های ساختاریافته به پوشه مستقل Modules/ تقسیم می‌شود. هر ماژول (مانند Modules/Inventory یا Modules/Financial) دارای کنترلرها، مدل‌ها، مایگريشن‌ها و روت‌های کاملاً اختصاصی و ایزوله خود است .

۴.۲. منع اتصالات مستقیم دیتابیس (No Physical Foreign Keys): ایجاد کلید خارجی فیزیکی (FK Constraint) بین جداول دو ماژول مختلف در دیتابیس ممنوع است. ارتباطات داده‌ای بین ماژول‌ها صرفاً از طریق آیدی منطقی (UUID) به صورت لوکال چک می‌شود.

۴.۳. ارتباطات درون‌پراسس: در فاز مونوپلیت، ارتباطات بلادرنگ و همگام (Synchronous) بین دو ماژول لاراول نباید از طریق درخواست‌های شبکه HTTP API انجام شود. ارتباطات باید صرفاً از طریق صدا زدن "Service Contracts / Interfaces" در محیط مموری هندل گردد تا پرفورمنس سیستم حفظ شده و معماری به کوپلینگ سخت دچار نشود.

=====

=====

۵. معماری فرانت‌آند (Next.js Micro-Frontend Strategy)

به منظور موازی‌سازی توسعه و جلوگیری از دست بردن تیم‌های مختلف در کدهای یکدیگر، فرانت‌آند پلتفرم بر اساس ساختار Next.js و استراتژی Micro-Frontend بنا می‌شود:

۵.۱. پروژه اصلی لایه پوسته: (Core Shell App)

یک مونو-ریپازیتوری مرکزی وظیفه روتینگ کلان، مدیریت قالب اصلی (Layout)، بارگذاری کامپوننت‌های مشترک UI (مانند منوها و نوتیفیکیشن‌ها) و حفظ Tenant Context را بر عهده دارد.

۵.۲. ماژول‌های مستقل فرانت‌آند: (Remote Micro-Apps)

هر ماژول ERP (مانند مالی یا انبار) به صورت یک پروژه مجزا یا پکیج کاملاً مستقل در ساختار کامپوننت‌های فرانت‌آند توسعه می‌یابد. یکپارچه‌سازی و بارگذاری این اجزا در لایه اجرا (Runtime) با استفاده از مکانیسم‌های استاندارد) مانند Module Federation یا (Next.js Multi-Zones انجام می‌شود تا توسعه‌دهندگان هر ماژول بتوانند بخش خود را بدون نیاز به دیپلوی کل پروژه فرانت‌آند، ارتقا دهند.

=====

=====

۶. جریان چرخه درخواست (Request Lifecycle Flow)

هر رکوئست ورودی از کلاینت به سمت پلتفرم HamarehERP، باید گام‌های امنیتی و زیرساختی زیر را طی کند:

۱. کلاینت (Next.js Application) رکوئست را همراه با توکن امنیتی JWT ارسال می‌کند.

۲. لایه گیت وی/میدل ور لاراول توکن را رمزگشایی کرده و Tenant Context (شناسه مستأجر و محدوده سازمانی) را استخراج و قفل می‌کند.

۳. شناسه مستأجر به سشن دیتابیس پورتگرس تزریق می‌شود (SET local app.current_tenant_id).

۴. رکوئست به ماژول مربوطه در لایه بک‌آند هدایت شده و بیزینس لاجیک با فیلترهای خودکار مدل‌ها (Global Scopes) و لایه امنیت پایگاه‌داده (RLS) اجرا می‌گردد.

۵. در صورت نیاز به تغییر داده در ماژول‌های دیگر، رویداد در جدول Outbox ثبت شده و پاسخ سریعاً به کلاینت بازمی‌گردد.

=====

=====

۷. خطوط قرمز معماری (Architectural Red Lines)

عدم رعایت هر یک از موارد زیر به معنای نقض کامل بلوپرینت پلتفرم بوده و کدهای مربوطه مردود خواهند شد :

- نوشتن بیزینس لاجیک در لایه فرانت‌آند یا کلاینت .
- خواندن یا نوشتن مستقیم در جداول دیتابیس یک ماژول توسط ماژول دیگر .
- دور زدن لایه احراز هویت متمرکز یا استفاده از متغیرهای سشن ناپایدار بدون شناسه مستأجر .
- استفاده از درخواست‌های شبکه داخلی HTTP API برای تعاملات همگام بین ماژول‌ها در فاز مونوپلیت .

=====

=====

۸. قانون نهایی

این سند بلوپرینت مبنای هماهنگی تیم‌های توسعه فرانت‌آند، بک‌آند و پایگاه‌داده سیستم HamarehERP است. هرگونه تغییر در لایه‌بندی‌ها، استراتژی تفکیک فیزیکی دیتابیس‌ها یا روش‌های ارتباطی ذکر شده در این سند، نیازمند تصویب رسمی و ثبت یک الحاقیه معماری (Architecture Amendment) در لایه حاکمیت پروژه خواهد بود .